



**tecnun**

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA  
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA  
Escuela Superior de Ingenieros • Ingeniarien Goi Mailako Eskola

## Examen Diciembre C++

---



*Informática II*

*Prof. Paul Bustamante / Jesus Paredes*

***Nombre:***

***Carné:***

---

## Índice

|                                                                              |   |
|------------------------------------------------------------------------------|---|
| .....                                                                        | 1 |
| 1. INTRODUCCIÓN.....                                                         | 1 |
| 2. EJERCICIO 1: ORDENAR PALABRAS (0.75).....                                 | 1 |
| 3. EJERCICIO 2: CÁLCULO DEL ÁREA Y PERÍMETRO DE UN POLÍGONO (1.75 PTS.)..... | 1 |
| 4. EJERCICIO 3: GESTIÓN UN GIMNASIO CON CLASES (2.0 PTS.) .....              | 3 |
| 5. EJERCICIO 4: GESTIÓN DE UN TALLER CON POO (3.0 PTS.) .....                | 6 |

### 1. Introducción

Recuerde que debe dejar los ejercicios en la carpeta “**C:\Temp\ExamInfor**”. Si no los deja allí, no se le podrá corregir el examen. Trate de realizar primero el ejercicio que crea que es más sencillo y deje el complicado para el final.

### 2. Ejercicio 1: Ordenar palabras (0.75)

Este ejercicio consiste en hacer un programa que pida dos nombres (o palabras) y los imprima de forma ordenada (ascendente o descendente, según desee el usuario). **NO** puede usar la función **strcmp**.

El programa deberá continuar hasta que el usuario decida salir.

```

E:\Tecnun\Examen2023\Ejer1\64\Debu...
**** Ordenando nombres ****
Dar primer nombre:?adolfo
Dar segundo nombre:?adan
Orden Ascendente o Descendente(A/D):?A
*****
Nombres Ordenados Ascendente:
adan
adolfo
Desea continuar o salir (c/s):?_

```

### 3. Ejercicio 2: Cálculo del área y perímetro de un polígono (1.75 Pts.)

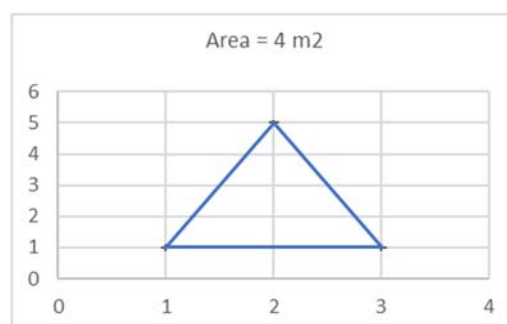
A Ud. le han pedido en su empresa de topografía que haga un programa que permita calcular el área y el perímetro de determinados terrenos, dadas sus coordenadas (en mt.), usando el método de proyecciones o trapecio. Este método sirve para cualquier polígono, regular o irregular.

Dicho método consiste en lo siguiente: dados “**n**” puntos (x e y), el área es el sumatorio de la **Ec.1**, teniendo en cuenta que el **último punto será el primer punto**, para cerrar el polígono. Recuerde que en C++ los índices empiezan en ‘0’. El perímetro es la suma de las distancias entre los puntos.

$$Area = \sum_{i=1}^n \frac{(y_i + y_{i+1}) * (x_{i+1} - x_i)}{2} \quad Ec.1$$

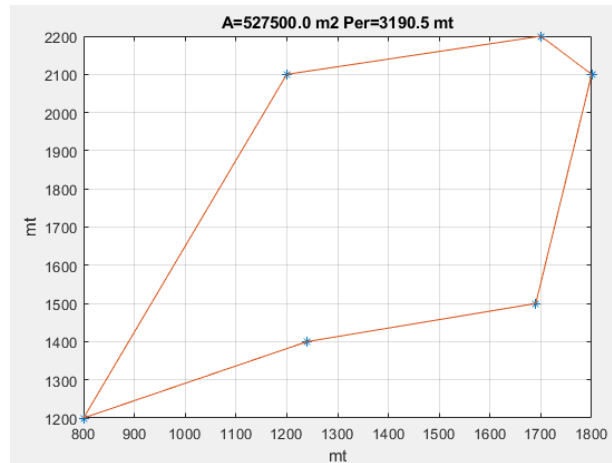
Veamos un ejemplo sencillo, un triángulo (3 vértices) de base 2 y altura 4, cuyas coordenadas son:

| X (mt) | Y (mt) |
|--------|--------|
| 1      | 1      |
| 2      | 5      |
| 3      | 1      |



El área es igual a  $((1+5)*(2-1) + (5+1)*(3-2) + (1+1)*(1-3)) / 2 = 4 \text{ m}^2$  (en rojo las Y y en verde las X, del sumatorio) En la última suma, usar el primer valor, para cerrar el polígono.

Otro ejemplo, un polígono de 6 vértices:



### Programa:

Para el desarrollo de este programa debe crear la siguiente estructura:

```
struct Vertice {
    double x, y;
};
```

Cada punto será representado por esta estructura.

El programa deberá mostrar un menú con dos opciones. La opción 1 será para hacer los cálculos, para lo cual deberá pedir primero el número de vértices del polígono a medir y luego los datos. Deberá **usar reserva dinámica** de memoria para el vector de estructuras:

```
Vertice *vec; //vector de estructuras (para reserva dinámica de memoria)
```

Deberá hacer dos funciones:

- Función **CalcularArea (1.0 Pts)**: pasarle un vector de estructuras tipo **Vertice**. Deberá devolver el cálculo del área: double **CalcularArea**(argumentos necesarios)
- Función **CalcularPerimetro (0.5 Pts)**: pasarle un vector de estructuras tipo **Vertice**. Deberá devolver el cálculo del perímetro: double **CalcularPerimetro**(argumentos necesarios).

```
E:\Tecnun\Examen2023\Ejer1\Debug\Ejer1.e...
** Empresa de Topografía **
1.Dar valores
2.Salir
Opc: ? 1
Dar vertices: ? 6
Dar el punto(x,y): ? 800 1200
Dar el punto(x,y): ? 1200 2100
Dar el punto(x,y): ? 1700 2200
Dar el punto(x,y): ? 1800 2100
Dar el punto(x,y): ? 1690 1500
Dar el punto(x,y): ? 1240 1400
El Area es: 527500 m2
El Perimetro es: 3190.51 mt

** Empresa de Topografía **
1.Dar valores
2.Salir
Opc: ?
```

**Nota:** Si no hace las funciones y no usa estructuras, no tendrá validez el ejercicio.

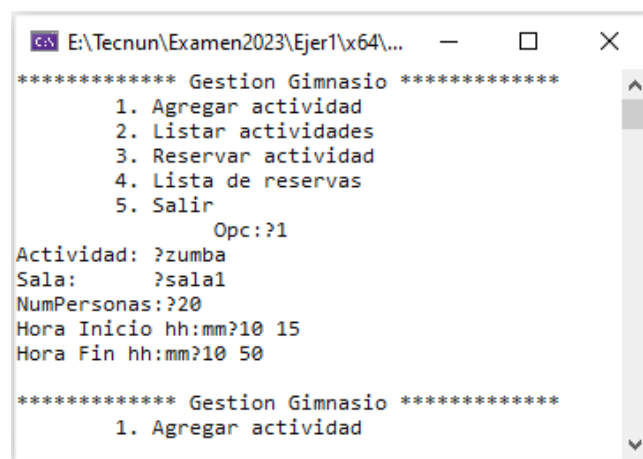
**Resto del ejercicio: 0.25 pts** (vector de estructuras, optimización de código, liberar memoria, etc)

#### 4. Ejercicio 3: Gestión un gimnasio con clases (2.0 Pts.)

A Ud. le han pedido que haga un programa para la gestión de un gimnasio, usando la POO. Dicho programa deberá tener las siguientes opciones:

- **Opcion1 (0.5 Pts.): Agregar actividad:** deberá pedir los siguientes datos: nombre de la actividad, nombre de la sala, número de personas (capacidad de la sala) y Hora de Inicio y Fin. Ver Figura 1.
- **Opcion2 (0.4 Pts.): Listar actividades:** deberá mostrar por pantalla todas las actividades agregadas en la opción 1, con todos sus datos. Ver Figura 2. A medida que se vayan haciendo reservas, el número de personas (capacidad de la sala) deberá ir disminuyendo.
- **Opcion3 (0.5 Pts.): Reservar una actividad.** El programa deberá pedir el nombre de la actividad a reservar y deberá mostrar todas las actividades con plazas libres (cuyo **número de personas sea > 0**) de ese día con dicho nombre. El usuario deberá elegir la actividad que desee y luego se le pedirá el número de carnet. El número de **personas de dicha actividad deberá disminuir**. Ver Figura 3.
- **Opcion4 (0.4 Pts.): Listado de reservas** de actividades. Ver Figura 4.
- **Opcion5:** Salir (liberando memoria reservada)

Por tema facilidad de programación se han omitido algunas cosas, como registrar al socio, comprobar que no reserva varias actividades a la misma hora, etc.



```
***** Gestion Gimnasio *****
1. Agregar actividad
2. Listar actividades
3. Reservar actividad
4. Lista de reservas
5. Salir
Opc: ?1
Actividad: ?zumba
Sala: ?sala1
NumPersonas: ?20
Hora Inicio hh:mm?10 15
Hora Fin hh:mm?10 50

***** Gestion Gimnasio *****
1. Agregar actividad
```

Figura 1 Agregar actividad

```

E:\Tecnun\Examen2023\Ejer1\x64\Debug\Ejer1.exe
***** Gestion Gimnasio *****
1. Agregar actividad
2. Listar actividades
3. Reservar actividad
4. Lista de reservas
5. Salir
Opc: ?2

***** Listado de actividades *****
      zumba      sala1      20      de 10:15 a 10:50
      gap        sala1      20      de 12:15 a 12:55
      bodypump   sala2      30      de 10:15 a 10:50
      bike       salaBike   20      de 12:10 a 12:50
      pilates    Master     50      de 18:30 a 19:15
      gap        sala2      20      de 20:10 a 20:50
      zumba      sala1      20      de 18:10 a 19:50
*****
***** Gestion Gimnasio *****

```

Figura 2 Listado de actividades

```

Seleccionar E:\Tecnun\Examen2023\Ejer1\x64\Debug\Ejer1.exe
Opc: ?2

***** Listado de actividades *****
      zumba      sala1      19      de 10:15 a 10:50
      gap        sala1      20      de 12:15 a 12:55
      bodypump   sala2      30      de 10:15 a 10:50
      bike       salaBike   20      de 12:10 a 12:50
      pilates    Master     50      de 18:30 a 19:15
      gap        sala2      19      de 20:10 a 20:50
      zumba      sala1      20      de 18:10 a 19:50
*****
***** Gestion Gimnasio *****

```

Listado de actividades después de haber hecho una reserva en **zumba** y **gap**

```

Seleccionar E:\Tecnun\Examen2023\Ejer1\x64\Debug\Ejer1...
***** Gestion Gimnasio *****
1. Agregar actividad
2. Listar actividades
3. Reservar actividad
4. Lista de reservas
5. Salir
Opc: ?3
Que actividad quiere: ?zumba
Encontrados:
[1] Actividad:      zumba      sala1      20      de 10:15 a 10:50
[2] Actividad:      zumba      sala1      20      de 18:10 a 19:50
Selecciona actividad (0 para no): ?1
Actividad seleccionada:
      zumba      sala1      de 10:15 a 10:50
Dar Carnet: ?9001

***** Gestion Gimnasio *****
1. Agregar actividad
2. Listar actividades
3. Reservar actividad

```

Figura 3 Reservar Actividad

```

E:\Tecnun\Examen2023\Ejer1\x64\Debug\Ejer1.exe
***** Gestion Gimnasio *****
1. Agregar actividad
2. Listar actividades
3. Reservar actividad
4. Lista de reservas
5. Salir
Opc: 4

***** Listado de Reservas *****
Carnet          Datos de la actividad
9001            zumba      sala1    de 10:15 a 10:50
9005            gap        sala2    de 20:10 a 20:50
*****
***** Gestion Gimnasio *****
1. Agregar actividad

```

Figura 4 Listado de Reservas de actividades

**Programa:**

Para la realización del programa, deberá crear las siguientes clases (con las variables y **SIN** cambiar el especificador de acceso, público o privado). Podrá poner los constructores que necesite y agregar más funciones.

|                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class Hora {     int hh, min; public:     Hora() { ; }     //más funciones si necesita };  class Reserva {     int carnet; //carnet socio     Actividad *lst; //puntero a la actividad public:     Reserva() {};     //más funciones si necesita }; </pre> | <pre> class Actividad {     char nombre[20]; //zumba,gap,bike,etc     char sala[20]; //sala1, master1, etc.     int numPersonas; //Ira decreciendo     Hora Ini, Fin; //Hora inicio y fin public:     Actividad() {};     //más funciones si necesita }; </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

En cuanto a **main**, a continuación, se da un esqueleto de cómo podría ser:

```

//incluir las clases
#define MAX_ACT 20
#define MAX_RES 100
int main()
{
    Actividad *lstAct[MAX_ACT]; //vector para guardar las actividades
    Reserva* lstResv[MAX_RES]; //vector para guardar las reservas
    //definir las variables necesarias

    while (true) {
        int opc = Menu();
        if (opc == 5) {
            //liberar memoria y salir
        }
        if (opc == 1) {

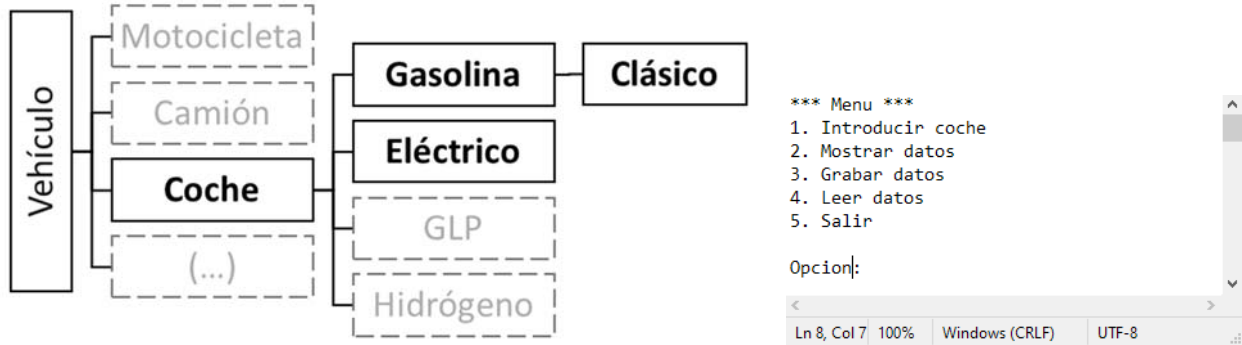
        }
    }
}

```

Función principal (**main**) liberando memoria y optimización del código (si tiene hecha al menos 2 opciones): **0.2 Pts.**

## 5. Ejercicio 4: Gestión de un taller con POO (3.0 Pts.)

Se le ha encargado realizar un pequeño programa para gestionar los vehículos que pasan por un taller. En una primera versión sólo funcionarán las clases coche, gasolina, clásico y eléctrico. No obstante, usted piensa en que en el futuro puede vender esta plataforma a talleres específicos para camiones, motos, etc. Por ello decide crear una clase más amplia (**vehículo**) en la que incluir todas las variables y define la siguiente jerarquía de clases:



Este programa deberá tener un menú que permita realizar las siguientes acciones:

(25%) La **opción 1** (Introducir coche): deberá preguntar el año de fabricación y la matrícula. Si el año de fabricación es anterior a 1970, se guardará automáticamente como clásico. Si no, se preguntará además si el coche es eléctrico o de gasolina.

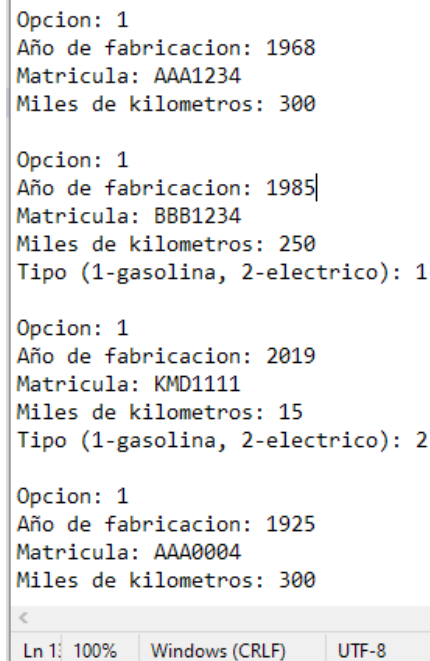
- Para este apartado deberá utilizar una **clase vehículo**, no sabe cómo será en el futuro, por lo que en esta versión solo tendrá:
  - Variables:** año y miles de kilómetros como variables protegidas; y matrícula como variable pública.
  - Funciones (máximo 3):** constructor por defecto, destructor por defecto y constructor de asignación (o una función de asignación en su defecto). Si lo desea puede combinar el constructor por defecto y el constructor de asignación en una sola función.

```

class vehiculo
{
protected:
    int year;    // Año de fabricación
    int km;     // miles de kilómetros
public:
    char plate[20]; // Matrícula
    // Constructor por defecto
    // Constructor o función de asignación
    // Destructor por defecto
};
  
```

- Las **clases coche, gasolina, eléctrico y clásico NO** tendrán variables propias, únicamente funciones públicas.





```
Opcion: 1
Año de fabricacion: 1968
Matricula: AAA1234
Miles de kilometros: 300

Opcion: 1
Año de fabricacion: 1985
Matricula: BBB1234
Miles de kilometros: 250
Tipo (1-gasolina, 2-electrico): 1

Opcion: 1
Año de fabricacion: 2019
Matricula: KMD1111
Miles de kilometros: 15
Tipo (1-gasolina, 2-electrico): 2

Opcion: 1
Año de fabricacion: 1925
Matricula: AAA0004
Miles de kilometros: 300
```

- Habrán dos listas donde guardar los coches registrados: un array de punteros de tipo coche **taller**, que almacenará todos los tipos de coche (clásicos incluidos); y un array dinámico **taller2** específica para los clásicos (ver definición en **main** a continuación). Es decir, que los clásicos estarán registrados por duplicado: en **taller** y en **taller2**.

```
void main()
{
    coche* taller[100]; //Almacena todos los coches (clásicos incluidos)
    clasico* taller2;   //Almacena sólo los clásicos
}
```

(25 %) La **opción 2** preguntará si desea imprimir todos los coches registrados (en **taller**) o sólo los clásicos (registrados en **taller2**).

- Para ello deberá desarrollar una función **void prt()** donde considere necesario. Cuando un objeto de tipo *Coche* llame a esta función, deberá imprimir el año, la matrícula y los kilómetros. Cuando un objeto de clase *Gasolina* llame a esta función deberá imprimir además que es de tipo gasolina. Del mismo modo, para los objetos de la clase *eléctrico*, deberá imprimir que es tipo eléctrico.
- En la clase clásico NO podrá incluir una función **void prt()**, sino que deberá crear una función específica **void prt\_clasico()** que imprimirá que es un *coche* clásico y los datos que imprime en la función **prt()** de la clase *coche* (año, la matrícula y los kilómetros).
- Tiene dos ejemplos a continuación de cómo debe funcionar el programa. Note que en la lista de la izquierda incluye los *clásicos* pero no se muestra el mensaje “Coche clásico”, mientras que en la lista derecha sí.

```

Opcion: 2

Mostrar lista coches(1) o clásicos(2)?:1

Año de fabricacion: 1968
Matricula: AAA1234
Miles de kilometros: 300

Coche electrico
Año de fabricacion: 1985
Matricula: BBB1234
Miles de kilometros: 250

Coche de gasolina
Año de fabricacion: 2019
Matricula: KMD1111
Miles de kilometros: 15

Año de fabricacion: 1925
Matricula: AAA0004
Miles de kilometros: 300

Ln 23, Col 100% Windows (CRLF) UTF-8

2. Mostrar

Mostrar lista coches(1) o clásicos(2)?:2

Coche clasico
Año de fabricacion: 1968
Matricula: AAA1234
Miles de kilometros: 300

Coche clasico
Año de fabricacion: 1925
Matricula: AAA0004
Miles de kilometros: 300

Ln 20, Col 100% Windows (CRLF) UTF-8

```

(25%) La **opción 3** permitirá grabar la lista de clásicos en un fichero. No es necesario que pregunte el nombre del fichero, guárdelo siempre en un fichero “**Registro.txt**”. Para guardar los datos, no es necesario que les dé ningún tipo de formato. A continuación, se muestra un ejemplo de cómo podrían guardarse.

```

1 1968
2 AAA1234
3 300
4 1925
5 AAA0004
6 300

```

- Para realizar este apartado deberá crear una función `void grabar(ofstream&)` dentro de la clase *clásico*. Para grabar los datos en el fichero, se creará en el main se creará un objeto `ofstream`. Mediante un bucle cada objeto del array de *clásicos* (**taller2**) deberá llamar a esta función y pasarle como argumento de entrada el objeto creado para escribir en el fichero.

(15%) La **opción 4** permitirá leer los datos de ese mismo fichero (“*Registro.txt*”) y almacenarlos en el array de clásicos. Antes de leer reinicie el contador de clásicos a cero (no es necesario que borre los elementos clásicos guardados en el array de coches).

(10 %) La **opción 5** permitirá salir del programa. Tenga en cuenta que debe liberar la memoria que haya reservado de forma dinámica y que debe cerrar los ficheros que haya abierto.

¡Suerte!!